

@state — inject

Appears in: [DL-01](#)

server/state.py:43-88

"The scenario injection @state applies a scenario by copying the base state and patching only the relevant fields." ... "the base state is copied on every injection (@state)."

@state — inject

server/state.py:43-88

```
43     def apply_scenario(self, name: str, preserve_scores: bool = False) -> dict:
44         path = SCENARIOS / f"{name}.json"
45         scenario = json.loads(path.read_text())
46
47         saved_scores = {}
48         if preserve_scores:
49             saved_scores = {
50                 s["supplier_id"]: s["reliability_score"]
51                 for s in self._current["suppliers"]
52             }
53
54         self._current = deepcopy(self._base)
55         self.agent_log = []
56         self.active_scenario = name
57
58         for key, overrides in scenario.get("overrides", {}).items():
59             if key == "inventory":
60                 for override_item in overrides:
61                     for item in self._current["inventory"]:
62                         if item["sku_id"] == override_item["sku_id"]:
63                             item.update(override_item)
64                             break
65             elif key == "suppliers":
66                 for override_item in overrides:
67                     for supplier in self._current["suppliers"]:
68                         if supplier["supplier_id"] == override_item["supplier_id"]:
69                             for k, v in override_item.items():
70                                 if k == "catalog" and isinstance(v, dict):
71                                     for sku_id, sku_data in v.items():
72                                         if sku_id in supplier["catalog"]:
73                                             supplier["catalog"][sku_id].update(sku_data)
74                                         else:
75                                             supplier["catalog"][sku_id] = sku_data
76                                 else:
77                                     supplier[k] = v
78                             break
79             elif key == "invoices":
80                 self._current["invoices"].extend(overrides)
81
82         if preserve_scores and saved_scores:
83             for supplier in self._current["suppliers"]:
84                 sid = supplier["supplier_id"]
85                 if sid in saved_scores:
86                     supplier["reliability_score"] = saved_scores[sid]
87
88         return scenario
```

What it shows: `apply_scenario()` deep-copies the pristine base state, then patches only the overridden inventory/supplier/invoice fields — so every run starts from the same clean baseline and runs never contaminate each other.

Source: `server/state.py:43-88` · image rendered by `build_snippets.py`